Final Report for Pokémon Battle Simulator

The program simulates a game of Pokémon battles between trainers. It will have a base class called **pokemon** in the **pokemon.h** extension file. It has as protected data members: a string for the name of the Pokémon, a string for the type, integer for the Health Points (HP), an integer for the ID, and a vector of pointers to structures called Attack. The structure **Attack** contains a string with the name of one attack of that particular Pokémon and an integer showing how much damage that attack causes to another Pokémon.

Additionally, it has the following functions: a default constructor and an overloaded constructor to determine the attributes of the Pokémon (its name, type, HP, ID, and a vector of pointers for the set of **Attacks** of that Pokémon). It will also have a copy constructor, copy assignment operator and destructor, the setters **void setName, void setType, void setHP, void setID, void setAttack** and the getters **string getName(), string getType(), int getHP(), unsigned int getID(), string getAttackName(unsigned int position), getAttackDamage(unsigned int position)**. I included the parameter position of the getters of the attacks because I want them to return values from a particular position in the vector of Attacks. For the implementation of the setters and getters of name, type, HP and ID, they are done the way learned in the course. The setter of the Attacks will call a new pointer to an Attack and set its values with the corresponding parameters of name and damage and push back in the vector of attacks. In addition, there is a function called **attack(const string & trainer_name,const string & _pokemon_name, int _enemy_HP)** that will invite the trainer, whose name is **trainer_name,** to use an attack from the set of that Pokémon (**_pokemon_name**) and its corresponding damage will subtract the HP (**_enemy_HP)** from the opponent Pokémon by the damage of the chosen attack and return the subtracted value. Finally, there will be a virtual function called

virtual void updateDamages(const string &_enemy_type,const string &_previous_enemy_type)

In the implementation from the derived classes **firetype.cpp**, **grasstype.cpp** and **watertype.cpp**, the virtual function will update the damages of each of the attacks based on the type of the enemies of the Pokémon. Depending on the type of the enemy(**const string &_enemy_type**), the damages will be increased (advantage) or decreased (disadvantage) by five. Then it will announce if that Pokémon is either in advantage or disadvantage and print its set of attacks with the updated damages. Since this function is used in battle in a sequence of disputes between Pokémon's, I expect the damages to

already have been updated previously if a Pokémon was already in battle. I included a command, in which, if the Pokémon was already in battle, it will set the damages back to their original values before updating again based on the type of the previous Pokémon they defeated (**_previous_enemy_type).** Additionally, for the derived classes, I included an overloading constructor without any additional member and a virtual destructor.

Additionally, it will have another class called **trainer** in the **trainer.h** extension file. It will have as private data members: a string for the name of the trainer, integer for the number of battles won and a **vector<pokemon*> team** for the team of Pokémon of the trainer, which will have pointers for the **pokemon** class. The public functions, implemented by the **trainer.cpp** file, are the overloaded constructor (in which only the name of the trainer and their number of badges will be defined), a copy constructor, copy assignment operator and destructor, a getter for the trainer name, a getter and a setter for the badge count, **void addPokemon(const pokemon &p)** for the Pokémon that is going to be added to the team vector input by the trainer by the push back according to their type (will be defined by a specific derived class). It will also, have a function **void removePokemon(const string &name)** to exclude a Pokémon, whose name is **name,** from the team by the pop back function, **void viewPokemon(const string & name)** to print the name, type and ID of the Pokémon, whose name is **name, void viewStats()** to print the name of the trainer, the number of battles won, and the whole team of Pokémon of the trainer, **int countPokemons()** to return the number of pokemons in the current team, **bool wasDefeated()** that determines if a trainer still has or not pokemons with HP greater than zero by summing the HP values. If that sum equals zero, this means that trainer was defeated. **pokemon* selectedPokemon()** that invites the used to select a pokemon whose HP is greater than zero (by inputting its name) and return the pointer to the chosen pokemon. That function will call itself again if the choice was not valid (the trainer does not have that pokemon, or it was defeated). It will print the message "*name of the pokemon*, I choose you!". Additionally, there will be a function called **void viewStats()** to print the trainer's current number of badges (increased by 1 by each won battle) and the current team of pokemons of that trainer. **void LoadingandSaving()** will ask if the trainer wants to save their current team of pokemons in an output file. If not it will clear the vector of the team. Else, it will output the pokemons attributes in the following way:

Pokemon .NameOfPokemon1,type,HP,ID

Attack .attack1,damage

Attack .attack2,damage

[...]

Pokemon .NameOfPokemon2,type,HP,ID

Attack .attack1,damage

Attack .attack2,damage

[...]

I included characters of pointers and commas so the getline commands do not store the values the wrong way for loading. Then it will use the getters for each attribute of that pokemon to print in the output file. Next, there will be a message asking the user to load an existing input file. If the file is open, it will open a while loop until the end of the file. Inside it, it will allocate the memory to a new pokemon and iput an identifier for a new pokemon (string for "Pokemon") then it will read the name, type, HP and ID separated by commas (will use getline) then I will allocate the new pokemon again depending on the input type for the corresponding derived class and then use the corresponding setters to each attribute and store the values. Then, there will be a nested while loop to store the attacks for that pokemon, in which I will use the identifier again. It will stop when identifier stops reading "Attack", meaning that pokemon no longer has attacks,pushes back the resulting pokemon in the vector of the pokemon team, and moves on to the next pokemon (identifier reads "Pokemon" again). Inside the nested while loop, it will have a getline for the name of each attack and then it will read its damage, then use the setter for attack with the values obtained and puch back to the vector of attacks. Finally, it prints out the current team of the trainer.

Also, the program will have a class called **battle** in the **battle.h** extension file. Which will call a battle between two trainers (the two parameters of the **dispute()** function, trainer1 and trainer2). In the **battle.cpp** file, there will be the implementation of the **dispute()** function, it will check the size of the team of Pokémon of each trainer to make sure it is not greater than 6, by calling the **countPokemons()** from each trainer. If it exceeds 6, it will jump outside the function and print a message saying which trainer had an excess of Pokémon. Otherwise, still inside the function, another message will announce the start of the battle and introduce the flag **last_loss** that will have specific values for each step of the value (0 for start, 1 when the last pokemon defeated belonged to trainer 1 and 2 if it belonged to the trainer 2). There will be a while loop that stops when one of the triners is defeated, **while(!(trainer1.wasDefeated()||trainer2.wasDefeated()))**. Inside it, there will be if statements for the flag **last_loss** to determine which trainer should call a new pokemon using **pokemon* selectedPokemon()** function. If that is the beginning of the battle (**last_loss=0)** then both should call new pokemons, whose HP's will be printed, and then, it will update their level of damages with the function **updateDamages(const string**

&_enemy_type,const string &_previous_enemy_type) since both pokemons are new **previous_enemy_type==** "none". If the last Pokémon of trainer 1 was defeated (**last_loss=1)**, **previous_enemy_type** will be assigned with the value of the type of that Pokémon, obtained by the getter of the type. Trainer 1 will call a new pokemon, their HP's will be printed, and then, it will update their level of damages with the function **updateDamages(const string &_enemy_type,const string &_previous_enemy_type)** for pokemon of trainer 2 and **updateDamages(const string &_enemy_type, "none")** for pokemon of trainer 1. Then it will open a nested while loop in which the function **attack(const string & trainer_name,const string & _pokemon_name, int _enemy_HP)** is called for each of the pokemons and their returned values are used in the setter for HP to update that value. Each time after that function is called, it will print the HPs for both pokemons. The nested while loop finishes when one of the HPs of one of the pokemons equals zero depending on which pokemon was it will print a meassage saying which pokemon cannot battle anymore and update the flag **last_loss.** When a trainer is defeated it will print a message announcing who the wineer was and increase their badge count by 1.

Instructions to use the game: in the main.cpp file write about a new pokemon, first writes its vector of attacks and push back in that vector. Then declasre the new pokemon with each of its attributes. To declare a new trainer, include only their name and badge count. To add pokemons to that trainer use the addpokemon function. To remove pokemon use the removepokemon function. To see more details about a particular pokemon in the team use the viewPokemon function. To declare a new battle first call it, then apply the two trainers to it. To save the current team of a trainer and load a new team use the LoadandSaving function.

Important note: my code is only able to build when I disable the **battle.cpp** file and use the implementation of its function in the **battle.h** file since it gives me a fatal error for excess of .cpp files.